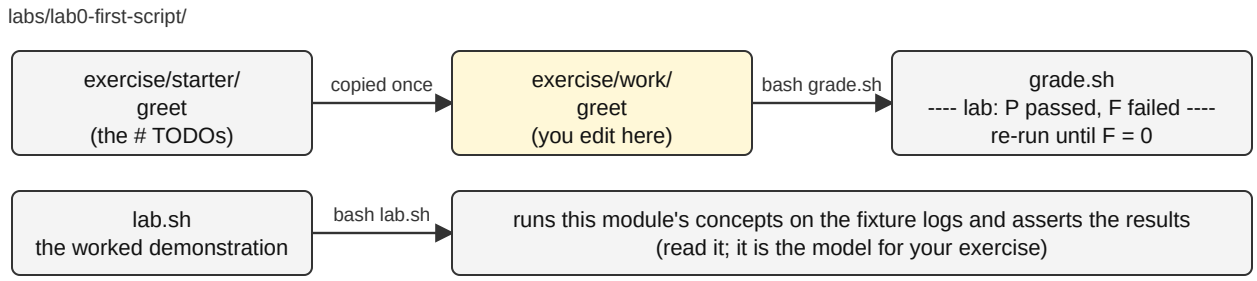
Lab 0 — Your First Script: greet

Topology



Environment Table

All paths are relative to the course folder.

File	What it is	You edit it?
`labs/lab0-first-script/exercise/TASK.md`	The short statement of the exercise	No
`labs/lab0-first-script/exercise/starter/greet`	The skeleton with three `# TODO` comments	No — the grader copies it
`labs/lab0-first-script/exercise/work/greet`	**Your** copy of the skeleton (created by the first `grade.sh` run)	**Yes**
`labs/lab0-first-script/exercise/grade.sh`	The grader: 6 checks	No
`labs/lab0-first-script/exercise/work/REFLECTION.md`	Your two reflection answers	**Yes** (you create it)

Objectives

- **Part 1:** Prepare the exercise folder and read the grader's first report.
- **Part 2:** Complete `greet`: the strict-mode header, the default argument, the `printf` line.
- **Part 3:** Verify with the grader until every check passes.
- **Part 4:** Write the two reflection answers.

Background / Scenario

Every script in this course — and every `ci/*.sh` script in the SecMLOps program — starts the same way: a shebang line that names the interpreter, a strict-mode header that makes mistakes fail loudly, and `printf` for output that another program might read. In this lab you build the smallest useful script, `greet`,

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.

the way the course builds all of them, and you meet the grader that will check your work in every later lab.

`greet [NAME]` prints `Hello, <name>!`. With no argument it prints `Hello, world!`. It must run under strict mode and pass `shellcheck` with no findings.

Required Resources

- The course folder, opened in a lab session (`docker run --rm -it -v "$PWD":/course -w /course bash-course:2.0`), or a shell where `bash tools-check.sh` reports `ready` (bash ≥ 4 and `shellcheck`).
- Module M00, steps S00.4–S00.9.
- A text editor. Documentation and AI assistance for lookups are allowed (say so in your reflection). Do **not** open `solution/`.

Part 1: Prepare the exercise folder

Step 1: Run the grader once.

```
cd labs/lab0-first-script/exercise
bash grade.sh
```

Expected: `---- lab0: 2 passed, 4 failed ----`. The first run copies `starter/greet` into `work/` and grades the copy. Each `FAIL` line shows what the check expected and what your script printed; the checks run top to bottom, so fix them in order.

Step 2: Open your working copy.

Open `work/greet` in your editor. It has three `# TODO` comments; each one quotes the exact line it needs. The first line, `#!/usr/bin/env bash`, is already there — it tells the system which program runs this file. Leave it.

Part 2: Complete `greet`

Step 1: Add the strict-mode header.

a. Directly under the shebang comment block, add the two header lines:

```
set -euo pipefail
IFS=$'\n\t'
```

b. What they do: `-e` stops the script at the first failing command; `-u` makes an unset variable an error instead of an empty string; `-o pipefail` makes a pipeline fail if any command in it fails; `IFS=$'\n\t'` stops a plain space from splitting values into several words.

Step 2: Read the first argument, with a default.

Replace `name="FIXME"` with:

```
name="${1:-world}"
```

`$1` is the first argument. `${1:-world}` means "use `$1`, or `world` if it is missing or empty". The quotes keep the value as one word whatever it contains.

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.

Step 3: Print the greeting.

Replace `printf 'FIXME\n'` with:

```
printf 'Hello, %s!\n' "$name"
```

`%s` is where the value goes; `\n` is the newline. The value is passed as a separate, quoted argument — never pasted into the format string.

Step 4: Try it by hand.

```
bash work/greet
bash work/greet Antonio
bash work/greet; echo "exit=$?"
```

Expected: `Hello, world!`, then `Hello, Antonio!`, then the greeting followed by `exit=0`.

Part 3: Verify with the grader

Step 1: Run the grader until it is green.

```
bash grade.sh
```

Expected: `---- lab0: 6 passed, 0 failed ----`. The six checks: default greeting; greets its argument; exits 0 on success; has an env-bash shebang; uses strict mode (the line `set -euo pipefail` is present); `greet` is shellcheck-clean.

Step 2: If a check fails.

Symptom	Cause	Fix
`uses strict mode` fails	The header line is misspelled or not at the start of a line	Copy `set -euo pipefail` exactly
`shellcheck-clean` fails with SC2086	`$name` is unquoted	Write `"$name"`
`has an env-bash shebang` fails	The first line was changed to `#!/bin/bash`	Restore `#!/usr/bin/env bash` — it finds bash wherever it is installed
Nothing prints, or `FIXME` prints	A placeholder line is still there	Replace it, do not add a line after it

Step 3: Reset if you want to start over.

```
rm -rf work && bash grade.sh
```

Part 4: Reflection

Create `work/REFLECTION.md` and answer both questions in your own words (3–5 sentences each). Name the mechanism; a restated rule does not pass.

Question 1: Why does the course use `printf` instead of `echo` for output that another program might read?

Question 2: What would break, and when, if you forgot the quotes around `"$name"`? Relate it to the SC2086 message you saw.

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.

Hand-in and grading

Hand in `work/greet`, `work/REFLECTION.md`, and your final grader output. **Pass** = the grader ends `0 failed` **and** both reflection answers name the mechanism. There is no limit on attempts.

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.